

Welcome to PhysX

Built May 18, 2026

Welcome to the NVIDIA PhysX SDK version 5! We are pleased to finally release this much anticipated update on GitHub, years after the release of version 4. PhysX has, in the meantime, become the primary physics engine of NVIDIA Omniverse, and for a while it was unclear if it will ever be available again for use in other applications and platforms.

PhysX has also become a key reference implementation of the similarly open source Pixar Universal Scene Description (USD) Physics standard available at [PixarAnimationStudios/USD](https://github.com/PixarAnimationStudios/USD) on GitHub. This informed the decision to return to the more permissive licensing terms used for PhysX 4. This means that all CPU source code is available under the simple BSD3 open source license, and NVIDIA GPU binaries are included at no cost.

We have also decided that our Flow and Blast libraries, while technically not depending on PhysX, should be part of a PhysX product family and licensed together. For that reason Flow is now bundled in the same GitHub repo and Blast will be added soon.

In terms of new features, PhysX 5 now includes former NVIDIA Flex library capabilities like finite element model based soft body dynamics, and liquid, cloth and inflatable objects using position based dynamics. We have also added a signed distance field collision feature, which lets the user perform collision detection using a voxelized version of the source mesh, thus avoiding having to create a convex decomposition. All these can only run on the GPU. In terms of new CPU features, PhysX 5 now lets the user define custom geometries, meaning cylinder shapes or implicit block based worlds can now be supported. Both CPU and GPU parallel computing performance for large simulations has been significantly improved.

Like already the change from PhysX 3 to 4, the public API doesn't have any massive changes. Nonetheless it will be a good idea to look at [Migrating From PhysX SDK 4.0 to 5.1](#) and possibly even the earlier migration guides, depending on which older SDK you are already familiar with. Users completely new to PhysX should probably just start reading this user's guide and then step through some of our very simple sample code we call snippets. More advanced demos are no longer bundled with the SDK; we encourage everyone to look at the physics demos in NVIDIA Omniverse for more advanced examples for what is. NVIDIA Omniverse is also where you should look for any content creation tools, we have made a big investment already to create the best possible physics toolset in Omniverse and we will continue to build it out. We will also soon release source code showing how to build a possibly user modified version of this PhysX SDK into a custom Omniverse extension.

About this User Guide

This Guide will help the reader to understand the features of the PhysX SDK and how to make use of them. The Guide is not exhaustive on the details of the API; the reader should refer to the PhysX API Reference Documentation for this at [C++ API](#). Furthermore this guide does not go into detail about the algorithms used in PhysX.

The [GitHub Discussions](#) page or the [Omniverse Discord Channel](#) is a good place to ask questions. Use the [GitHub Issues](#) to report bugs.

A brief overview of PhysX

PhysX is a library for representing three dimensional worlds made of discrete entities named actors which can in turn be composed of multiple shapes. PhysX lets the user create and destroy such actors, and tracks their explicit or proximity based interactions. Actors can either be static, be moved around by the user, or be moved by PhysX according to the laws of classical mechanics. PhysX' dynamics simulation capability includes support for collision, joints and actuation using maximal and/or reduced coordinates. Furthermore, the world may be queried by the user using a number of different tools ranging from simple ray-casts to sweep and overlap tests. PhysX provides extensions for special purpose functionality such as vehicle simulation.

PhysX is designed to be robust, high performance, scalable, portable, as well as easy to integrate and use. These capabilities make PhysX suitable as a foundation technology for game engines and other real time simulation systems.

It is important to note that PhysX does not run any code on GPUs by default. However PhysX can be configured to take advantage of CUDA capable GPUs, which provides a performance benefit proportional to the arithmetic complexity of a scene. GPU acceleration extensions are provided as an optional binary DLL. See [GPU Simulation](#) for details on how to enable GPU acceleration. Even without GPU acceleration, PhysX is fully multithreaded and SIMD-accelerated to take full advantage of modern multi-core CPUs. As mentioned above, PhysX 5 adds a number of GPU exclusive features like soft bodies. These features will not work unless the SDK is configured to run on a GPU.

World and Objects

The basic concepts of the world within a PhysX simulation are easy to describe:

- › The PhysX world comprises a collection of Scenes, each containing objects called Actors;

- › Each Scene defines its own reference frame encompassing all of space and time;
- › Actors in different Scenes do not interact with each other;
- › Characters and vehicles are complex specialized objects made from Actors;
- › Actors have physical state : position and orientation; velocity or momentum; energy; etc,
- › Actor physical state may evolve over time due to applied forces, constraints such as joints or contacts, and interactions between Actors.

Rendering and Visualization

3d rendering will of course be a vital technology to visualize the simulated world in almost every application. At the minimum the user needs to take the updated state of any moved visible actors and update them in the respective rendering representation. Production quality graphics and audio are outside the scope of PhysX; at NVIDIA we of course use Omniverse to visualize everything. Our example programs come with rudimentary built-in visualization you can use as a guide for writing your own code, and we provide batch APIs for rapidly retrieving the updated transforms of very many objects in one call. Omni-PVD, built on Omniverse, provides simulation recording, visualization and telemetry for diagnostic purposes. See [Omniverse Visual Debugger](#) for details.

Copyright © 2022-2026, NVIDIA Corporation.

Last updated on May 18, 2026.